

Core Language Changes for NB Comments at the February, 2020 (Prague) meeting

References in this document reflect the section and paragraph numbering of document [WG21 N4849](#).

US028. Define grammar for "punctuator"

1. Change 5.4 [lex.ptoken] paragraph 1 as follows:

Each preprocessing token that is converted to a token (5.6 [lex.token]) shall have the lexical form of a keyword, an identifier, a literal, **or** an operator, ~~or a~~ **or** punctuator.

2. Change the grammar in 5.6 [lex.token] as follows:

token:

identifier
keyword
literal
~~*operator*~~
~~*punctuator*~~
operator-or-punctuator

3. Change 5.11 [lex.key] paragraph 2 as follows:

Furthermore, the alternative representations shown in Table 6 for certain operators and punctuators (5.5 [lex.digraph]) are reserved and shall not be used otherwise.

4. Change 5.12 [lex.operators] paragraph 1 as follows:

The lexical representation of C++ programs includes a number of preprocessing tokens ~~which~~ **that** are used in the syntax of the preprocessor or are converted into tokens for operators and punctuators:

preprocessing-operator: one of

%: %:%:

~~***preprocessing-op-or-punc***~~ ***operator-or-punctuator: one of***

{	}	[	]	#	##	(	)	
<:	:>	<%	%>	%:	%:%:	;	:	...
new	delete	?	::	.	*	->	->*	~
!	+	-	*	/	%	^	&	
=	+=	-=	*=	/=	%=	^=	&=	=
==	!=	<	>	<=	>=	<=>	&&	
<<	>>	<<=	>>=	++	--	,		
and	or	xor	not	bitand	bitor	compl		
and_eq	or_eq	xor_eq	not_eq					

preprocessing-op-or-punc:

preprocessing-operator

operator-or-punctuator

Each ~~preprocessing-op-or-punct~~ **operator-or-punctuator** is converted to a single token in translation phase 7 (5.2 [lex.phases]).

US033. Allow "import" inside linkage-specifications [dcl.link]

1. Change the grammar in 6.6 [basic.link] paragraph 1 as follows:

```
translation-unit
top-level-declaration-seqopt
global-module-fragmentopt module-declaration top-level-declaration-seqopt private-module-fragmentopt

top-level-declaration-seq:
top-level-declaration
top-level-declaration-seq top-level-declaration

top-level-declaration:
module-import-declaration
declaration
```

2. Change 6.6 [basic.link] paragraph 2 as follows:

A token sequence beginning with `exportopt module` and not immediately followed by `::` is never interpreted as ~~the declaration of~~ a ~~top-level-declaration~~.

[Note: [P1857R3](#) deletes this paragraph, which supersedes this change. It is to be applied only if P1857R3 is not adopted.]

3. Change the grammar and paragraph 1 of 10.4 [module.global] as follows:

```
global-module-fragment:
    module ; top-level-declaration-seqopt
```

[Note: Prior to phase 4 of translation, only preprocessing directives can appear in the ~~top-level-declaration-seq~~ (15.5 [cpp.glob.frag]). —end note]

4. Change 10.4 [module.global] paragraph 4 as follows:

A declaration `D` in a global module fragment of a module unit is *discarded* if `D` is not decl-reachable from ~~any top-level-declaration in the top-level-declaration-seq of the translation-unit~~ **translation-unit**. [Note: A discarded declaration...

5. Change the grammar in 10.5 [module.context] as follows:

```
private-module-fragment:
    module : private ; top-level-declaration-seqopt
```

6. Change 10.6 [module.reach] paragraph 3 as follows:

During the implicit instantiation of a template whose point of instantiation is specified as that of an enclosing specialization (13.8.4.1 [temp.point]), the instantiation context is the union of the instantiation context of the enclosing specialization and, if the template is defined in a module interface unit of a module *M* and the point of instantiation is not in a module interface unit of *M*, the point at the end of the ~~top-level-declaration-seq~~ of the primary module interface unit of *M* (prior to the *private-module-fragment*, if any).

7. Change 13.8.4.1 [temp.point] paragraph 7 as follows:

A specialization for a function template, a member function template, or of a member function or static data member of a class template may have multiple points of instantiations within a translation unit, and in addition to the points of instantiation described above,

- for any such specialization that has a point of instantiation within the ~~top-level-declaration-seq~~ of the ~~translation-unit~~ **translation-unit**, prior to the *private-module-fragment* (if any), the point after the ~~top-level-declaration-seq~~ of the *translation-unit* is also considered a point of instantiation, and
- for any such specialization...

8. Change the grammar in 9.1 [dcl.pre] paragraph 1 as follows:

declaration:

block-declaration
nodeclspec-function-declaration
function-definition
template-declaration
deduction-guide
explicit-instantiation
explicit-specialization
export-declaration
linkage-specification
namespace-definition
empty-declaration
attribute-declaration
module-import-declaration

9. Change 10.3 [module.import] paragraph 1 as follows:

A *module-import-declaration* shall only appear at global namespace scope. In a module unit, all *module-import-declarations* shall precede all other ~~top-level-declarations~~ in the ~~top-level-declaration-seq~~ of the *translation-unit* and of the *private-module-fragment* (if any). The optional *attribute-specifier-seq* appertains to the *module-import-declaration*.

10. Add the following as a new paragraph after 9.11 [dcl.link] paragraph 3:

Every implementation shall provide for linkage to functions written in the C programming language, "C", and linkage to C++ functions, "C++". [Example: ... —end example]

A *module-import-declaration* shall not be directly contained in a *linkage-specification*. A *module-import-declaration* appearing in a linkage specification with other than C++ language linkage is conditionally-supported with implementation-defined semantics.

US041. Enforce correspondence of containing objects for pointer values / aliasing

Change 6.7.3 [basic.life] paragraph 8 as follows:

If, after the lifetime of an object has ended and before the storage which the object occupied is reused or released, a new object is created at the storage location which the original object occupied, a pointer that pointed to the original object, a reference that referred to the original object, or the name of the original object will automatically refer to the new object and, once the lifetime of the new object has started, can be used to manipulate the new object, if: **the original object is transparently replaceable (see below) by the new object.**

An original object is transparently replaceable by a new object if:

- ...
- neither the original object nor the new object is a potentially-overlapping subobject (6.7.2 [intro.object]); **and**
- **neither the original object nor the new object are subobjects or both are subobjects, and**
- **the (unique) object (if any) that contains the original object as a subobject (6.7.2 [intro.object]) is transparently replaceable by the (unique) object (if any) that contains the new object.**

[Example:...

CA104. Clarify declaration matching and partial ordering requiring substitution into constraints

1. Add the following as a new paragraph at the end of 13.5.2 [temp.constr.decl]:

When determining whether a given introduced *constraint-expression* *C1* of a declaration in an instantiated specialization of a templated class is equivalent (13.7.6.1 [temp.over.link]) to the corresponding *constraint-expression* *C2* of a declaration outside the class body, *C1* is instantiated. If the instantiation results in an invalid expression, the *constraint-expressions* are not equivalent. [Note: This can happen when determining which member template is specialized by an explicit specialization declaration. —end note] [Example:

```
template <class T> concept C = true;
template <class T> struct A {
    template <class U> U f(U) requires C<typename T::type>; // #1
    template <class U> U f(U) requires C<T>;                // #2
};
```

```
template <> template <class U> U A<int>::f(U u) requires C<int> { return u; } // OK, specializes #2
```

Substituting `int` for `T` in `C<typename T::type>` produces an invalid expression, so the specialization does not match #1. Substituting `int` for `T` in `C<T>` produces `C<int>`, which is equivalent to the *constraint-expression* for the specialization, so it does match #2. —end example]

2. Change 13.9.1 [temp.inst] paragraph 17 as follows:

The *type-constraints* and *requires-clause* of a template specialization or member function are not instantiated along with the specialization or function itself, even for a member function of a local class; substitution into the atomic constraints formed from them is instead performed as specified in 13.5.2 [temp.constr.decl] and 13.5.1.2 [temp.constr.atomic] when determining whether the constraints are satisfied **or as specified in 13.5.2 [temp.constr.decl] when comparing declarations**. [Note: The satisfaction of constraints

CA107. Parameter mapping for non-dependent entities

US117. Comparing types and type-constraints

1. Change 13.5.1.2 [temp.constr.atomic] paragraph 1 as follows:

An *atomic constraint* is formed from an expression *E* and a mapping from the template parameters that appear within *E* to template arguments **involving that are formed via substitution during constraint normalization in the declaration of a constrained entity (and, therefore, can involve the unsubstituted template parameters of the constrained entity)**, called the *parameter mapping* (13.5.2 [temp.constr.decl]). [Note: Atomic constraints are formed by constraint normalization (13.5.3 [temp.constr.normal]). *E* is never a logical AND expression (7.6.14 [expr.log.and]) nor a logical OR expression (7.6.15 [expr.log.or]). —end note]

2. Change 13.5.1.2 [temp.constr.atomic] paragraph 2 as follows:

Two atomic constraints, *e1* and *e2*, are *identical* if they are formed from the same *expression* and **if, given a hypothetical template *A* whose template-parameter-list consists of template-parameters corresponding and equivalent (13.7.6.1 [temp.over.link]) to those mapped by the parameter mappings of the expression, a template-id naming *A* whose template-arguments are the targets of the parameter mappings are equivalent according to the rules for expressions described in 13.7.6.1 [temp.over.link], mapping of *e1* is the same (13.6 [temp.type]) as a template-id naming *A* whose template-arguments are the targets of the parameter mapping of *e2*.** [Note: The comparison of parameter mappings of atomic constraints operates in a manner similar to that of declaration matching with alias template substitution (13.7.7 [temp.alias]).] [Example:

```
template <unsigned N> constexpr bool Atomic = true;
template <unsigned N> concept C = Atomic<N>;
template <unsigned N> concept Add1 = C<N + 1>;
template <unsigned N> concept AddOne = C<N + 1>;
template <unsigned M> void f()
```

```
requires Add1<2 * M>;
template <unsigned M> int f()
requires AddOne<2 * M> && true;
```

```
int x = f<0>(); // OK, the atomic constraints from concept C in both fs are Atomic<N>
// with mapping similar to  $N \mapsto 2 * M + 1$ 
```

```
template <unsigned N> struct WrapN;
template <unsigned N> using Add1Ty = WrapN<N + 1>;
template <unsigned N> using AddOneTy = WrapN<N + 1>;
template <unsigned M> void g(Add1Ty<2 * M> *);
template <unsigned M> void g(AddOneTy<2 * M> *);
```

```
void h() {
    g<0>(nullptr); // OK, there is only one g
}
```

—end example]

This similarity includes the situation where a program is ill-formed, no diagnostic required, when the meaning of the program depends on whether two constructs are equivalent, and they are functionally equivalent but not equivalent. [Example:

```
template <unsigned N> void f2()
requires Add1<2 * N>;
template <unsigned N> int f2()
requires Add1<N * 2> && true;
void h2() {
    f2<0>(); // ill-formed, no diagnostic required:
// required determination of subsumption between atomic constraints that are
// functionally equivalent but not equivalent
}
```

—end example] —end note]

US115. Hidden non-template friends need a *requires*-clause

1. Add the following after 3.20 [defns.signature]:

3.21

[defns.signature.friend]

signature

⟨non-template friend function with trailing *requires*-clause⟩ name, parameter-type-list (9.3.3.5 [dcl.fct]), enclosing class, and trailing *requires*-clause (9.3 [dcl.decl])

2. Add the following after 3.21 [defns.signature.templ]:

3.23

[defns.signature.templ.friend]

signature

⟨friend function template with constraint involving enclosing template parameters⟩ name, parameter-type-list (9.3.3.5 [dcl.fct]), return type, enclosing class, *template-head*, and trailing *requires*-clause

3. Change 13.7.4 [temp.friend] paragraph 9 as follows:

A non-template friend declaration ~~shall not have~~ with a *requires*-clause shall be a definition. A friend function template with a constraint that depends on a template parameter from an enclosing template shall be a definition. Such a constrained friend function or function template declaration does not declare the same function or function template as a declaration in any other scope.